Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Факультет электронно-информационных систем
Кафедра ИИТ

Отчет по лабораторной работе № 3

Специальность ПО13

Выполнил:
А.С.Потапчук,
Студент группы ПО13
Проверил:
А.А.Крощенко
Доцент кафедры ИИТ
«____» _____ 2026 г.

Брест 2026

Цель работы: *приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.*

Вариант 8

Общее задание

• Прочитать задания, взятые из каждой группы, соответствующей одному из трех основных типов паттернов;

• Определить паттерн проектирования, который может использоваться при реализации задания. Пояснить свой выбор;

• Реализовать фрагмент программной системы, используя выбранный паттерн. Реализовать все необходимые дополнительные классы.

Задание 1

8)Торговый автомат с возможностью выдачи любого выбранного товара (шоколадные батончики, чипсы, пакетированные соки и т.д.)

Код программы:

```
from abc import ABC, abstractmethod

class Product(ABC):

    @abstractmethod
    def get_name(self):
        pass

class Chocolate(Product):

    def get_name(self):
        return "Chocolate bar"

class Chips(Product):

    def get_name(self):
        return "Chips"

class Juice(Product):

    def get_name(self):
        return "Juice"

class ProductFactory:

    @staticmethod
    def create_product(product_type):

        if product_type == "chocolate":
            return Chocolate()

        elif product_type == "chips":
            return Chips()

        elif product_type == "juice":
            return Juice()

        else:
            return None

class VendingMachine:

    def order_product(self, product_type):

        product = ProductFactory.create_product(product_type)

        if product:
            print("Dispensing:", product.get_name())
        else:
            print("Product not available")

def main():

    vending_machine = VendingMachine()
```

```
    while True:

        print("\nSelect product:")
        print("1 - Chocolate")
        print("2 - Chips")
        print("3 - Juice")
        print("0 - Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            vending_machine.order_product("chocolate")

        elif choice == "2":
            vending_machine.order_product("chips")

        elif choice == "3":
            vending_machine.order_product("juice")

        elif choice == "0":
            print("Goodbye!")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Select product:
1 - Chocolate
2 - Chips
3 - Juice
0 - Exit
Enter choice: 1
Dispensing: Chocolate bar

Select product:
1 - Chocolate
2 - Chips
3 - Juice
0 - Exit
Enter choice: 2
Dispensing: Chips

Select product:
1 - Chocolate
2 - Chips
3 - Juice
0 - Exit
Enter choice: 0
Goodbye!
Press any key to continue . . .
```

Задание 2

ДУ телевизора. Реализовать иерархию телевизоров для конкретных производителей и иерархию средств дистанционного управления. Телевизоры должны иметь присущие им атрибуты и функции. ДУ имеет набор функций для изменения текущего канала, увеличения/уменьшения громкости, включения/выключения телевизора и т.д. Эти функции должны отличаться для различных устройств ДУ.

Код программы :

```
from abc import ABC


class TV(ABC):
    def __init__(self):
        self.power = False
```

```python
        self.volume = 10
        self.channel = 1

    def turn_on(self):
        self.power = True
        print("TV is ON")

    def turn_off(self):
        self.power = False
        print("TV is OFF")

    def volume_up(self):
        if self.volume < 100:
            self.volume += 1
        print(f"Volume: {self.volume}")

    def volume_down(self):
        if self.volume > 0:
            self.volume -= 1
        print(f"Volume: {self.volume}")

    def channel_up(self):
        self.channel += 1
        if self.channel > 999:
            self.channel = 1
        print(f"Channel: {self.channel}")

    def channel_down(self):
        self.channel -= 1
        if self.channel < 1:
            self.channel = 999
        print(f"Channel: {self.channel}")


class SonyTV(TV):
    def __init__(self):
        super().__init__()
        print("Sony TV created")


class SamsungTV(TV):
    def __init__(self):
        super().__init__()
        print("Samsung TV created")


class RemoteControl:
    def __init__(self, tv):
        self.tv = tv

    def toggle_power(self):
        if self.tv.power:
            self.tv.turn_off()
        else:
            self.tv.turn_on()

    def volume_up(self):
        self.tv.volume_up()

    def volume_down(self):
        self.tv.volume_down()

    def channel_up(self):
        self.tv.channel_up()

    def channel_down(self):
        self.tv.channel_down()


class BasicRemote(RemoteControl):
    def mute(self):
        self.tv.volume = 0
        print("Muted")


class AdvancedRemote(RemoteControl):
    def set_channel(self, number):
        if 1 <= number <= 999:
            self.tv.channel = number
            print(f"Channel set to {number}")
        else:
            print("Invalid channel number")
```

```python
def main():
    print("Bridge pattern demo\n")

    tv = SonyTV()
    print()

    remote = BasicRemote(tv)
    remote.toggle_power()
    remote.volume_up()
    remote.volume_up()
    remote.channel_up()
    remote.mute()
    remote.toggle_power()

    print("\n" + "-" * 50 + "\n")

    tv2 = SamsungTV()
    print()

    adv_remote = AdvancedRemote(tv2)
    adv_remote.toggle_power()
    adv_remote.set_channel(42)
    adv_remote.volume_up()
    adv_remote.channel_down()
    adv_remote.toggle_power()


if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Struct pattern

Sony TV created

TV is ON
Volume: 11
Volume: 12
Channel: 2
Muted
TV is OFF

--------------------------------------------------

Samsung TV created

TV is ON
Channel set to 42
Volume: 11
Channel: 41
TV is OFF
Press any key to continue . . .
```

8) Вспомогательная библиотека для работы с текстовыми файлами. Должны быть предусмотрены следующие функции: чтение одного или нескольких файлов, изменение файла(ов), отмена последней выполненной операции (одной в случае простой единичной операции или нескольких в случае сложной операции), последовательное выполнение нескольких операций.

Код программы:

```python
import os

class Command:
    """The abstract base class for all file operations."""
    def execute(self):
        pass

    def undo(self):
        pass


class EditCommand(Command):
    """Command to modify the content of one or multiple files."""
    def __init__(self, file_paths, new_text):
        self.file_paths = file_paths
        self.new_text = new_text
```

```python
        self.old_contents = {}

    def execute(self):
        for path in self.file_paths:
            if os.path.exists(path):
                with open(path, 'r', encoding='utf-8') as f:
                    self.old_contents[path] = f.read()
            else:
                self.old_contents[path] = None

            with open(path, 'w', encoding='utf-8') as f:
                f.write(self.new_text)
        print(f"Modified files: {', '.join(self.file_paths)}")

    def undo(self):
        for path, content in self.old_contents.items():
            if content is None:
                if os.path.exists(path):
                    os.remove(path)
            else:
                with open(path, 'w', encoding='utf-8') as f:
                    f.write(content)
        print(f"Rollback completed for: {', '.join(self.file_paths)}")


class MacroCommand(Command):
    """Groups multiple operations into a single complex action."""
    def __init__(self, commands):
        self.commands = commands

    def execute(self):
        for cmd in self.commands:
            cmd.execute()

    def undo(self):
        for cmd in reversed(self.commands):
            cmd.undo()

class FileEditor:
    def __init__(self):
        self._history = []

    def read_files(self, file_paths):
        results = {}
        for path in file_paths:
            if os.path.exists(path):
                with open(path, 'r', encoding='utf-8') as f:
                    results[path] = f.read()
            else:
                results[path] = "Error: File not found"
        return results

    def execute_action(self, command):
        command.execute()
        self._history.append(command)

    def undo_last(self):
        if not self._history:
            print("History is empty.")
            return
        command = self._history.pop()
        command.undo()


if __name__ == "__main__":
    editor = FileEditor()
    test_files = ["sample1.txt", "sample2.txt"]

    print("--- Step 1: Single Operation ---")
    edit_cmd = EditCommand(test_files, "Hello World!")
    editor.execute_action(edit_cmd)

    print("\n--- Step 2: Complex Operation (Macro) ---")
    cmd1 = EditCommand(["sample1.txt"], "Data Chunk A")
    cmd2 = EditCommand(["sample2.txt"], "Data Chunk B")
    macro = MacroCommand([cmd1, cmd2])
    editor.execute_action(macro)

    print("\n--- Current File States ---")
    print(editor.read_files(test_files))

    print("\n--- Step 3: Undoing Macro ---")
```

```python
    editor.undo_last()

    print("\n--- Step 4: Undoing First Operation ---")
    editor.undo_last()
```

Результат работы программы:

```
--- Step 1: Single Operation ---
Modified files: sample1.txt, sample2.txt

--- Step 2: Complex Operation (Macro) ---
Modified files: sample1.txt
Modified files: sample2.txt

--- Current File States ---
{'sample1.txt': 'Data Chunk A', 'sample2.txt': 'Data Chunk B'}

--- Step 3: Undoing Macro ---
Rollback completed for: sample2.txt
Rollback completed for: sample1.txt

--- Step 4: Undoing First Operation ---
Rollback completed for: sample1.txt, sample2.txt
Press any key to continue . . .
```

Вывод: *приобрела навыки применения паттернов проектирования при решении практических задач с использованием языка Python.*